



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ _____ Фундаментальные науки

КАФЕДРА _____ Прикладная математика

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ
НА ТЕМУ:

*Реализация, обучение и тестирование
математической модели искусственной
нейронной сети*

Студент _____
ФН2-51Б
(Группа)

(Подпись, дата)

С. И. Тихомиров

(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

А. А. Лазарев

(И. О. Фамилия)

2025 г.

Оглавление

Введение	3
1. Постановка задачи	4
2. Историческая справка	4
3. Теоретические сведения	5
3.1. Классификация нейронных сетей	5
3.2. Устройство нейросетей	7
4. Практическая часть	10
4.1. Обучающая выборка	10
4.2. Реализация нейросети и ее особенности	11
4.3. Сравнительный анализ результатов	13
Заключение	19
Список использованных источников	20

Введение

В современную эпоху стремительного развития технологий и увеличения объемов данных искусственные нейронные сети (ИНС) занимают ключевое место в области искусственного интеллекта. Эти модели способны адаптироваться, обучаться и анализировать информацию, что делает их незаменимым инструментом для решения сложных задач. Их применение охватывает практически все аспекты современной жизни: от распознавания изображений и обработки естественного языка до высокоточной медицинской диагностики, прогнозирования климатических изменений и разработки автомобилей с самостоятельным управлением. Кроме того, ИНС находят применение в творческих областях, таких как генерация музыки и искусства, что еще раз подчеркивает их универсальность и значимость.

Актуальность исследований в области ИНС обусловлена несколькими ключевыми факторами. Во-первых, современное количество данных требует современных методов анализа и обработки информации. Именно нейронные сети способны эффективно обрабатывать большие объемы данных, обеспечивая при этом высокую точность их анализа.

Во-вторых, внедрение ИНС в прикладные науки и области инженерии способствует заметному прогрессу в таких направлениях, как автоматизация производственных процессов, экономика, финансовый сектор и социальные исследования.

В-третьих, ускоренное развитие алгоритмов обучения и увеличение мощности вычислительной техники делают возможным решение задач, которые еще несколько лет назад считались нерешаемыми. Например, автоматический перевод, распознавание образов или игра в сложные логические игры теперь реализуемы благодаря ИНС.

1. Постановка задачи

Цель: Написать программу на C++, моделирующую ИНС с заданными параметрами. Обучить сеть для распознавания образов на каком-либо наборе данных из открытого доступа и проверить эффективность распознавания. Допускается использовать сторонние библиотеки для реализации.

Задачи: исследовать строение ИНС и подобрать оптимальные параметры для наилучшего результата.

2. Историческая справка

Идея создания ИНС появилась задолго до возникновения современных компьютеров. Ее истоки можно найти в середине XX века, когда ученые начали искать способы моделирования процессов мышления и обучения человека.

Первый значительный шаг в этой области был сделан Уорреном Маккаллоком и Уолтером Питтсом в 1943 году. Они предложили математическую модель искусственного нейрона, который мог выполнять простейшие логические операции. Эта работа заложила основу для будущих исследований в области искусственного интеллекта и нейронных сетей.

В 1958 году Фрэнк Розенблатт разработал персептрон — первую обучающуюся нейронную сеть. Персептрон был способен решать задачи классификации на основе обучающих примеров. Однако модель столкнулась с ограничениями: Марвин Мински и Сеймур Пейперт в 1969 году показали, что однослойные персептроны не способны справляться с линейно неразделимыми задачами, что привело к временному снижению интереса к нейросетям.

В 1980-х годах произошел так называемый "ренессанс" нейронных сетей, когда были предложены методы обратного распространения ошибки. Работа Джеффри Хинтона, Дэвида Румельхарта и Рональда Уильямса стала ключевым событием этого периода. Алгоритм обратного распространения позволил обучать многослойные нейронные сети, значительно расширив их возможности.

В 2000-х годах развитие вычислительной техники, в частности графических процессоров (GPU), и доступ к большим объемам данных дали новый импульс исследованиям в области ИНС. Возникли глубокие нейронные сети (deep learning), которые стали основой для современных достижений в компьютерном зрении, обработке естественного языка и других областях.

В 2020 году компания OpenAI представила одну из самых прорывных языковых

моделей GPT-3 (Generative Pretrained Transformer 3). А в конце ноября 2022-го года в свет вышла ChatGPT и спустя 2 месяца пробивает отметку в 100 миллионов пользователей, получив статус самого быстрорастущего онлайн-сервиса за всю историю.

Сегодня искусственные нейронные сети продолжают активно развиваться. Современные модели, такие как трансформеры и генеративные сети, позволяют решать сложные задачи с высокой степенью точности и эффективности. Кроме того, исследования в области биологически вдохновленных сетей и квантовых вычислений открывают новые перспективы для будущего развития ИНС.

3. Теоретические сведения

3.1. Классификация нейронных сетей

Начнем с понятия нейросеть. Что это такое? Нейронная сеть — это последовательность нейронов, соединенных между собой синапсами. Структура нейронной сети пришла в мир программирования напрямую из биологии. Благодаря такой структуре, машина обретает способность анализировать и даже запоминать различную информацию. Нейронные сети также способны не только анализировать входящую информацию, но и воспроизводить ее из своей памяти. Иными словами, нейросеть это машинная интерпретация мозга человека, в котором находятся множество нейронов передающих информацию в виде электрических импульсов.

Нейросети могут решить огромное множество задач в разных отраслях деятельности. Для удобства выделяют несколько базовых типов задач, для решения которых могут использоваться нейросети.

- 1) **Классификация.** Для распознавания лиц, эмоций, типов объектов: например, квадратов, кругов, треугольников. Также для распознавания образов, то есть выбора конкретного объекта из предложенного множества: например, выбор квадрата среди треугольников.
- 2) **Регрессия.** Для определения возраста по фотографии, составления прогноза биржевых курсов, оценки стоимости имущества и других задач, требующих получения в результате обработки конкретного числа.
- 3) **Прогнозирования временных рядов.** Для составления долгосрочных прогнозов на основе динамического временного ряда значений. Например, нейросети применяются для предсказания цен, физических явлений, объема потребления и других показателей. По сути, даже работу автопилота Tesla можно отнести к процессу прогнозирования временных рядов.

- 4) **Кластеризация.** Для изучения и сортировки большого объема неразмеченных данных в условиях, когда неизвестно количество классов на выходе, то есть для объединения данных по признакам. Например, кластеризация применяется для выявления классов картинок и сегментации клиентов.
- 5) **Генерация.** Для автоматизированного создания контента или его трансформации. Генерация с помощью нейросетей применяется для создания уникальных текстов, аудиофайлов, видео, раскрашивания черно-белых фильмов и даже изменения окружающей среды на фото.

Для решения данных задач существуют целые классы нейросетей, которые отличаются архитектурой, особенностями функционирования и сферами применения. Рассмотрим их.

1) **Персептрон**

Фундаментальная форма нейронных сетей, созданная в 1957 году Фрэнком Розенблаттом. Этот инструмент представляет собой одиночный или множественный набор нейронов, способных принимать решения, основанные на входных данных. Важно понимать, что персептрон стал отправной точкой для развития более сложных и эффективных структур.

2) **Многослойные Персептроны (MLP)**

Это тип нейронных сетей с несколькими слоями нейронов. Они используют определенные функции для обработки данных и обучаются с учетом ошибок, чтобы делать точные выводы. Их применяют для классификации и предсказания.

3) **Сверточные нейронные сети (CNN)**

Они специализируются на обработке данных с сетчатой структурой, таких как изображения. Эти сети выделяют характеристики и сжимают данные, что полезно для компьютерного зрения.

4) **Рекуррентные нейронные сети (RNN)**

Они подходят для обработки последовательных данных, где важен контекст, например, в обработке текстов или временных рядов. RNN применяются в задачах машинного перевода, распознавания речи и анализа временных рядов.

5) **Генеративно-сопоставительные сети (GAN)**

Сети, которые создают данные, неотличимые от реальных, и трансформеры - архитектура для обработки текстов, которая работает быстрее и эффективнее.

6) **Трансформеры**

Трансформеры — это архитектура нейросетей, разработанная для обработки последовательностей данных, таких как текст. В отличие от RNN, трансфор-

меры обрабатывают всю последовательность одновременно, используя механизм внимания для выделения значимых элементов. Это делает трансформеры более эффективными и позволяет обрабатывать большие объемы данных параллельно. Трансформеры нашли широкое применение в задачах обработки естественного языка, включая машинный перевод, ответ на вопросы и генерацию текста. Примером успешного применения трансформеров является модель GPT (Generative Pre-trained Transformer), разработанная OpenAI.

3.2. Устройство нейросетей

Начнем с введения понятия нейрон. Нейрон — это вычислительная единица, которая получает информацию, производит над ней простые вычисления и передает ее дальше. Они делятся на три основных типа: входной (синий), скрытый (красный) и выходной (зеленый). Также есть нейрон смещения и контекстный нейрон. В том случае, когда нейросеть состоит из большого количества нейронов, вводят термин слоя. Соответственно, есть входной слой, который получает информацию, n скрытых слоев (обычно их не больше 3), которые ее обрабатывают и выходной слой, который выводит результат (рис.2). У каждого из нейронов есть два основных параметра: входные данные (input data) и выходные данные (output data). В случае входного нейрона: $\text{input}=\text{output}$. В остальных, в поле input попадает суммарная информация всех нейронов с предыдущего слоя, после чего, она нормализуется, с помощью функции активации и попадает в поле output. Для удобства входные данные нормализуют при помощи функции активации. То есть, если на входе будет большое число, пропустив его через функцию активации, получится выход в нужном диапазоне. Функций активации достаточно много поэтому рассмотрим самые основные: Линейная, Сигмоид (Логистическая) и Гиперболический тангенс. Главные их отличия — это диапазон значений.

Также существует такое понятие, как синапс — это связь между двумя нейронами. У синапсов есть 1 параметр — вес. Благодаря ему, входная информация изменяется, когда передается от одного нейрона к другому. Допустим, есть 3 нейрона, которые передают информацию следующему. Значит есть 3 веса, соответствующие каждому из этих нейронов. У того нейрона, у которого вес будет больше, та информация и будет доминирующей в следующем нейроне (пример — смешение цветов (рис.1)). На самом деле, совокупность весов нейронной сети или матрица весов — это своеобразный мозг всей системы. Именно благодаря этим весам, входная информация обрабатывается и превращается в результат.

Математическая модель нейрона Маккаллока - Питтса разработана по аналогии

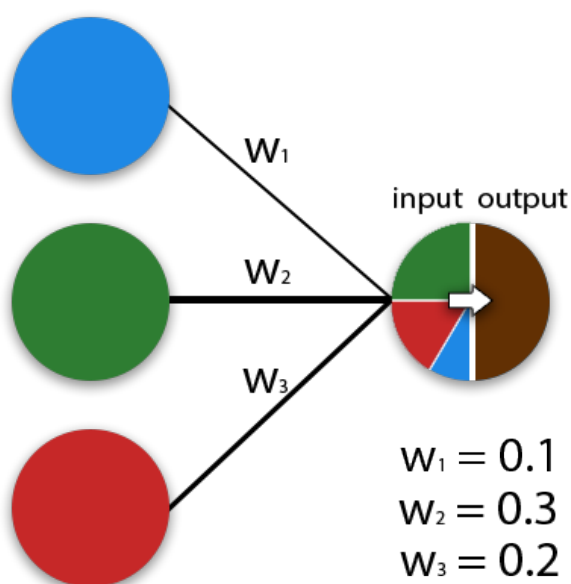


Рис. 1. Пример синаптической связи

с биологическими нервными клетками и выглядит следующим образом(рис.2):

X — входные данные — сигналы, поступающие к нейрону;

W — вес — эквивалент синаптической связи, представленный в виде действительного числа, на которое умножается значение входного сигнала для определения степени взаимосвязи отдельных нейронов. w является положительным числом, если один узел возбуждает другой, или отрицательным, если один узел подавляет другой. Узлы с более высокими значениями веса имеют большее влияние на другие узлы.

H — тело нейрона — показатель накопленной взвешенной суммы, полученной в результате умножения значений входящих сигналов на вес;

Y — выход нейронной сети — функция, получаемая в результате обработки входных сигналов.

На простом примере рассмотрим принцип работы нейросети. В данном примере изображена часть нейронной сети (рис. 3), где буквами l обозначены входные нейроны, буквой H — скрытый нейрон, а буквой w — веса, которые связаны данными формулами:

$$H_{input} = (l_1 w_1) + (l_2 w_2)$$

$$H_{output} = f_{activation} H_{input}$$

Из формулы видно, что входная информация — это сумма всех входных данных, умноженных на соответствующие им веса. Тогда дадим на вход 1 и 0. Пусть $w_1 = 0.4$ и $w_2 = 0.7$ Входные данные нейрона H_1 будут следующими:

$$H_1 = 1 \cdot 0.4 + 0 \cdot 0.7 = 0.4$$

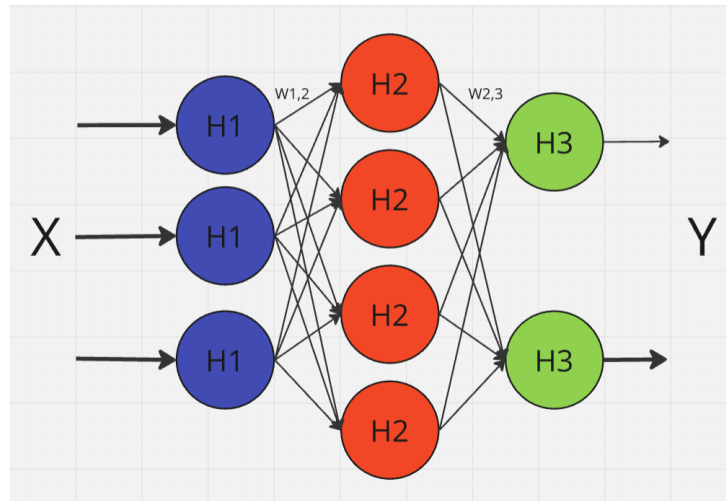


Рис. 2. Схема многослойного персептрона

Теперь когда у нас есть входные данные, мы можем получить выходные данные, подставив входное значение в функцию активации. Теперь, когда у нас есть выходные данные, мы передаем их дальше. И так, мы повторяем для всех слоев, пока не дойдем до выходного нейрона. Запустив такую сеть в первый раз мы увидим, что ответ далек от правильно, потому что сеть не натренирована. Чтобы улучшить результаты мы будем ее тренировать.

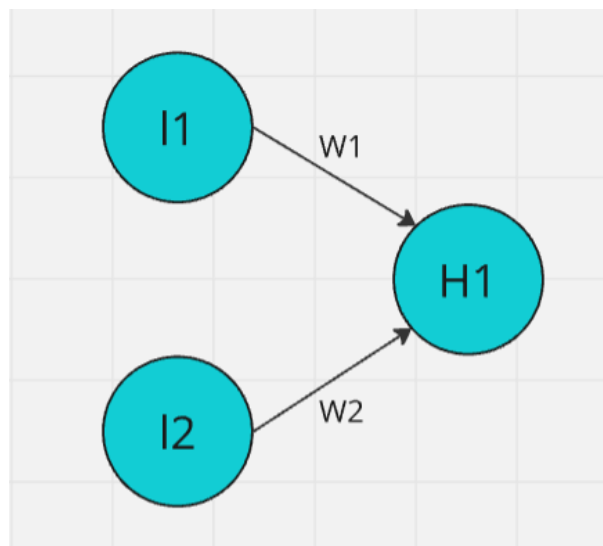


Рис. 3. Принцип работы нейросети

В ходе обучения нейросети на каждой эпохе формируется ошибка, которая должна идти на спад. Ее можно вычислить разными способами, но рассмотрим лишь три основных:

Mean Squared Error (далее MSE)

$$\varepsilon = \frac{\sum_{n=1}^k (i_n - a_n)^2}{n}$$

Root MSE

$$\varepsilon = \sqrt{\frac{\sum_{n=1}^k (i_n - a_n)^2}{n}}$$

Arctan.

$$\varepsilon = \frac{\sum_{n=1}^k \arctan^2(i_n - a_n)}{n}$$

Здесь нет какого-либо ограничения на использование, как в функции активации, главное выбрать метод, который будет приносить наилучший результат. Стоит лишь учитывать, что каждый метод считает ошибки по-разному. У Arctan, ошибка, почти всегда, будет больше, так как он работает по принципу: чем больше разница, тем больше ошибка. У Root MSE будет наименьшая ошибка, поэтому, чаще всего, используют MSE, которая сохраняет баланс в вычислении ошибки.

4. Практическая часть

4.1. Обучающая выборка

Для реализации нейросети был выбран датасет MNIST. Акроним «NIST» означает «National Institute of Standards and Technology» (Национальный институт стандартов и технологий) — учреждение, первоначально собравшее эти данные. А буква «М» означает «модифицированный», поскольку данные были подвергнуты предварительной обработке, чтобы упростить применение алгоритмов машинного обучения. Набор данных MNIST содержит отсканированные изображения рукописных цифр и ассоциированные с ними метки, описывающие, какая цифра от 0 до 9 изображена. Это одна из самых простых задач классификации, поэтому набор широко используется для тестирования в исследованиях по глубокому обучению. Он сохраняет популярность, хотя для современных методов задача не представляет труда. Джеффри Хинтон назвал его «дрозофилой машинного обучения», имея в виду, что он позволяет специалистам по машинному обучению исследовать свои алгоритмы в контролируемых лабораторных условиях, как биологи изучают фруктовых мушек [1].

В работе будет использован датасет в измененном виде. В файле .csv формата потоком записаны матрицы 28×28 элементов визуально напоминающие образ числа от 0 до 9. Всего обучающая выборка состоит из 60000 примеров и тестирование проводится на выборке из 10000.

4.2. Реализация нейросети и ее особенности

В работе реализована нейросеть типа многослойного персептрона с одним скрытым слоем. Так как условная "картинка" с числом имеет размер 28×28 пикселей то входной слой будет состоять из 784 нейронов. В скрытом слое число нейронов может изменяться, но оставим его на отметке в 256 нейронов. Нейросеть должна предсказывать число, изображенное на "картинке" поэтому выходной слой будет состоять из 10 нейронов. Также на входной и скрытый слой добавлено по одному нейрону смещения. Особенность этого типа нейронов заключается в том, что его вход и выход всегда равняются 1 и они никогда не имеют входных синапсов. Нейрон смещения нужен для того, чтобы иметь возможность получать выходной результат, путем сдвига графика функции активации вправо или влево. Значение нейрона смещения возьмем равное единице. Для работы с векторами и матрицами был создан класс *Matrix* с нужным набором функций.

Рассмотрим весь процесс работы нейросети на одной итерации. После загрузки данных из файла датасета, создается матрица весов, состоящая из случайных чисел от 0 до 1. Затем матрица весов умножается на вектор входных нейронов. Далее к полученной взвешенной сумме применяем функцию активации. Теперь инициализируем матрицу весов синапсов между скрытым и выходным слоем. Затем выполняются те же процедуры для нахождения значений выходных нейронов. По другому это называется передача вперед (Forward pass), то есть последовательная передача информации от входных нейронов к выходным.

Обучение нейросети будет проводится методом обратного распространения. Сначала введем определение ошибки (E) — это разность между эталонным значением и предсказанным нейросетью. Определим функцию ошибки:

$$E(w, b) = \sum_{i=0}^9 (d_i - y_i)^2 \rightarrow \min,$$

где d_i — эталонный ответ, y_i — предсказанный ответ.

Задачу минимизации функции $E(w, b)$ будем решать методом градиентного спуска. Итерационная формула для весов синапсов:

$$w = w - \alpha \nabla E, \quad \text{где } \nabla E = \sum_{i=0}^9 \left(\frac{\partial E}{\partial w_{ij}} + \frac{\partial E}{\partial b_{1j}} \right)$$

где α — скорость обучения (константа в интервале от 0 до 1). С каждой итерацией скорость обучения будет уменьшаться.

Теперь рассмотрим пример, где два нейрона выходного слоя вычисляются через нейроны скрытого слоя (рис.4).

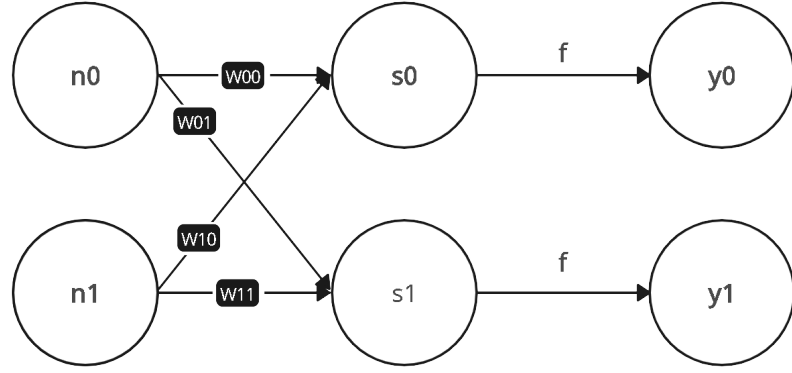


Рис. 4. Вычисление нейронов выходного слоя

Посчитаем $\frac{\partial E}{\partial w_{00}} = \frac{\partial E}{\partial s_0} \frac{\partial s_0}{\partial w_{00}} + \frac{\partial E}{\partial s_1} \frac{\partial s_1}{\partial w_{00}}$

$$s_0 = w_{00}n_0 + w_{01}n_1 + b_{10}, \quad \frac{\partial s_0}{\partial w_{00}} = n_0, \quad \frac{\partial s_0}{\partial b_{10}} = 1,$$

$$\frac{\partial E}{\partial s_0} = \frac{\partial (d_0 - y_0)^2}{\partial s_0} = 2(d_0 - y_0) \left(-\frac{\partial y_0}{\partial s_0} \right), \quad \frac{\partial y_0(s_0)}{\partial s_0} = f'(s_0), \quad d_0 - y_0 = \varepsilon_0.$$

Учитывая промежуточные вычисления получим:

$$w_{1ij} = w_{1ij} - 2\alpha \varepsilon_j f'(s_j^2) n_i^1,$$

$$b_{10} = b_{10} - 2\alpha \varepsilon_0 f'(s_0).$$

По аналогии с предыдущим примером вычислим частную производную ошибки по весам синапсов между входным и скрытым слоем:

$$\frac{\partial E}{\partial w_{000}} = \frac{\partial E}{\partial s_0^1} \frac{\partial s_0^1}{\partial w_{000}}, \quad s_0^1 = \sum_{m=0}^M w_{0m0} n_m^0 + b_{00},$$

$$\frac{\partial E}{\partial s_0^1} = \frac{\partial E}{\partial n_0^1} \frac{\partial n_0^1}{\partial s_0^1}, \quad n_0^1 = f(s_0^1),$$

$$\frac{\partial E}{\partial n_0^1} = \sum_{m=0}^M \frac{\partial E}{\partial s_m^2} \frac{\partial s_m^2}{\partial n_0^1}, \quad \frac{\partial s_k^2}{\partial n_0^1} = w_{10k}, \quad \frac{\partial E}{\partial s_k^2} = -2\varepsilon_k f'(s_k^2)$$

$$\frac{\partial E}{\partial w_{000}} = n_0^0 f'(s_0^1) \sum_{k=0}^N -2\varepsilon_k f'(s_k^2) w_{10k}$$

Обозначим за дельту нейронов

$$\delta_k^1 = f'(s_0^1) \sum_{l=0}^L \varepsilon_l f'(s_l^2) w_{10l}.$$

Тогда формула корректировки весов будет:

$$w_{000} = w_{000} + \tilde{\alpha} \cdot n_0^0 \cdot \delta_0^1$$

$$b_{00} = b_{00} + \tilde{\alpha} \cdot \delta_0^1$$

Так обновляются все веса на каждой итерации. Сначала обновляются веса синапсов между выходным и скрытым слоями, затем между скрытым и входным слоями. От такой последовательности хода от конца к началу метод и получил название обратного распространения.

Обучение будет производиться стохастическим методом, то есть при нахождении новых значений весов, их матрица сразу же обновляется. В конце обучения веса будут записаны в файл `Weights.txt`, который будет использован для тестирования нейросети. Обучать нейросеть планируется до ста процентов точности, но с ограничением по времени в 20 эпох.

Для реализации нейронной сети было создано несколько классов и функций:

- 1) Класс `Matrix` для хранения двумерных массивов, а также функции инициализации, заполнения матрицы произвольного размера случайными числами, а также функции суммирования векторов и умножения обычной и транспонированной матрицы на вектор-столбец.
- 2) Класс `ActivateFunction`, где заданы активационные функции и их производные. Пользователь сам выбирает какую активационную функцию использовать для обучения.
- 3) Класс `NetWork`, где реализованы функции работы нейросети: функция инициализации матрицы весов, функции прямого и обратного распространения, функции для обновления записи и сохранения матрицы весов.

Конфигурация нейросети будет считываться из файла `Config.txt`, где будет записано количество нейронов на каждом слое.

4.3. Сравнительный анализ результатов

В работе были задействованы самые распространенные функции активации:

- 1) Логистическая функция (рис.5)

$$f(x) = \frac{1}{1 + e^{-x}}, \quad E(y) = (0, 1),$$

- 2) Функция `LeakyReLU` (рис.6)

$$f(x) = \max(0.01x, x), \quad E(y) = (-\infty, +\infty)$$

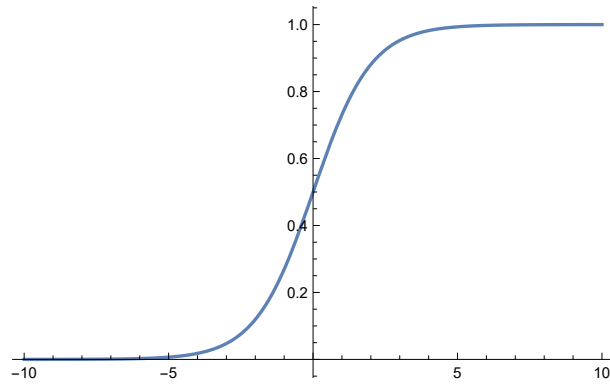


Рис. 5. График логистической функции

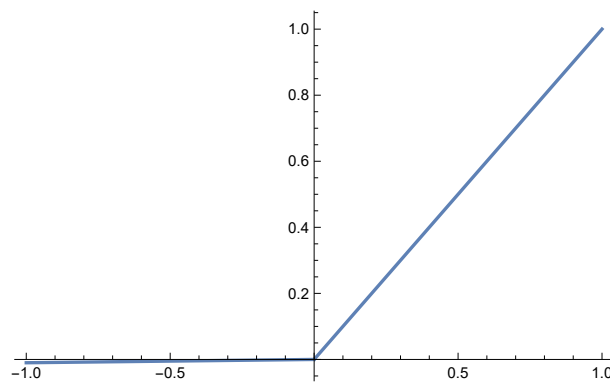
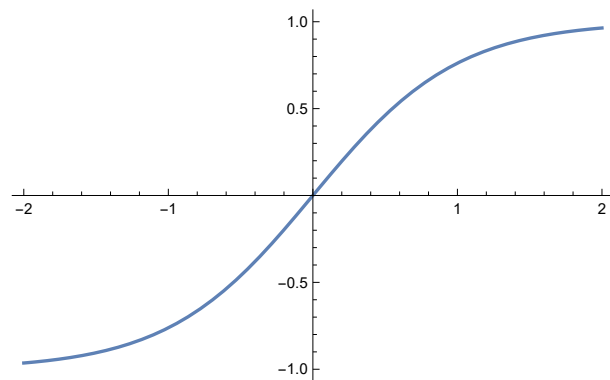


Рис. 6. График функции Leaky ReLU

3) Функция $\text{th } x$ (рис.7)

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad E(y) = (-1, 1)$$

Рис. 7. График функции $\text{th } x$

Для удобства и увеличения скорости обучения значения обучающей выборки были нормированы, так чтобы они находились в интервале от 0 до 1. В связи с этим надо доопределить функции LeakyReLU и $\text{th } x$.

Область значений функции LeakyReLU: $(-\infty, +\infty)$, а нейроны могут принимать значения в промежутке от 0 до 1. Поэтому для ограничения данных сверху добавим утечку:

$$f(x) = \begin{cases} 0.01x, & \text{если } x < 0; \\ x, & \text{если } 0 \leq x \leq 1; \\ 1 + 0.01(x - 1), & \text{если } x > 1. \end{cases}$$

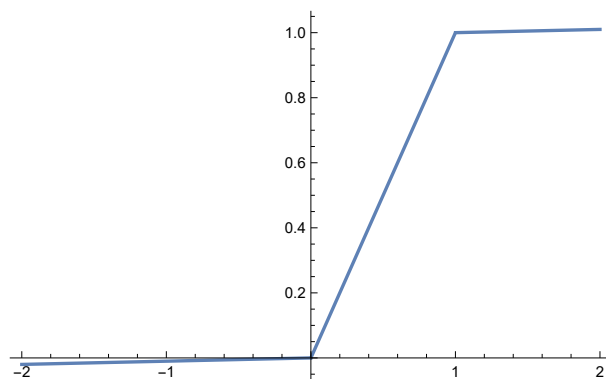


Рис. 8. График функции modReLU

Область значений $\text{th } x$: $(-1, 1)$, что не подходит под условия данной задачи. Поэтому добавим утечку снизу:

$$f(x) = \begin{cases} \text{th } x, & \text{если } x \geq 0; \\ 0.01 \text{th } x, & \text{если } x < 0. \end{cases}$$

Рассмотрим сильные и слабые стороны каждой функции активации. Сигмоида по-

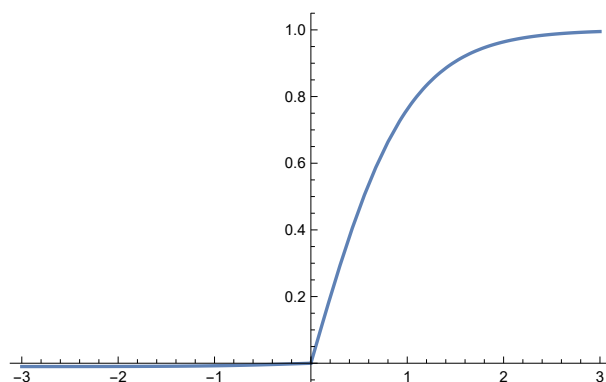


Рис. 9. График модифицированной функции $\text{th } x$

лезна в задачах бинарной классификации, так как всегда отображает входные данные на значение между 0 и 1, что можно интерпретировать как вероятность принадлежности к одному из двух классов. Еще одним преимуществом сигмоидной функции является ее плавный градиент, что позволяет легко вычислять градиенты при

обратном распространении и обновлять веса нейронной сети с помощью градиентного спуска. Основной недостаток сигмоиды заключается в его производной.

$$f'(x) = \left(\frac{1}{1 + e^{-x}} \right)' = \frac{e^{-x}}{(1 + e^{-x})^2} = f^2(x)(1 + e^{-x} - 1) = f^2(x)\left(\frac{1}{f(x)} - 1\right) = f(x)(1 - f(x))$$

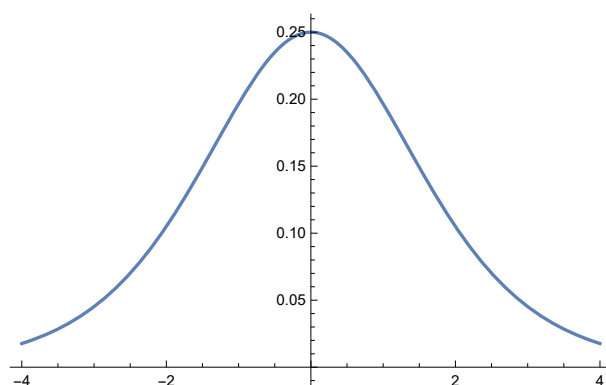


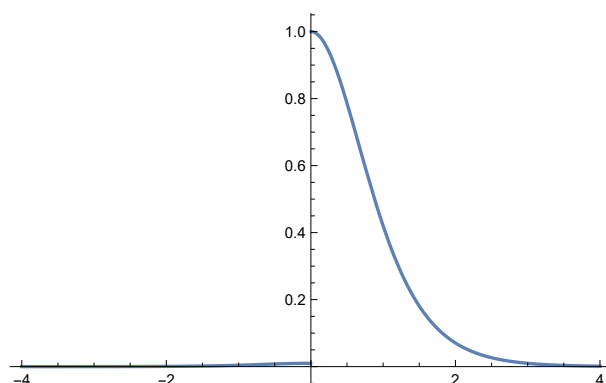
Рис. 10. График производной функции сигмоида

Сигмоида имеет ограниченный диапазон значимых значений градиента, который находится между -3 и 3 (рис. 10). За пределами этого диапазона функция становится более плоской, что приводит к очень маленьким градиентам. Данное свойство приводит к проблеме исчезающего градиента (The Vanishing Gradient Problem), когда сеть с трудом обучается, поскольку градиенты приближаются к нулю. Кроме того, значения функции не симметричны вокруг нуля, поэтому выход всех нейронов будет иметь одинаковый знак. Эта особенность делает обучение нейронной сети более сложным и нестабильным.

В результате обучения нейросети получено 74.75% точности предсказанных ответов на тестовой выборке.

В классическом понимании гиперболический тангенс похож на сигмоидную функцию, но его диапазон значений находится между -1 и 1, что делает его более симметричным вокруг нуля. Это позволяет использовать его в более широком диапазоне задач, таких как классификация с несколькими классами. Но для данной задачи область значений функции была изменена. Но даже с такой модификацией градиент гиперболического тангенса также может столкнуться с проблемой исчезающего градиента за пределами диапазона значимых значений (рис. 11).

$$f'(x) = \begin{cases} 1 - \text{th}^2 x, & \text{если } x \geq 0; \\ 0.01(1 - \text{th}^2 x), & \text{если } x < 0. \end{cases}$$

Рис. 11. График производной функции $\text{th } x$

Учитывая выше описанные особенности функции $\text{th}(x)$ получаем значение точности больше, чем при тестировании логистической функции, а именно 85.32%.

Решение проблемы исчезающего градиента является использование ReLU (Rectified Linear Unit).

$$\text{ReLU}(x) = \max(0, x)$$

Помимо решения проблемы с градиентом, ReLU также превосходит в простоте и скорости вычислений, что делает ее полезной для глубоких нейронных сетей с большим количеством параметров. Однако ReLU также имеет некоторые недостатки. Один из основных недостатков ReLU заключается в том, что он страдает от проблемы "умирающего ReLU" (dying ReLU [2]), когда большое количество нейронов в сети может стать неактивным и больше никогда не активироваться во время обучения. Это происходит, когда вход в нейрон отрицательный и градиент становится нулевым, в результате чего веса нейрона больше никогда не обновляются. Еще одним недостатком ReLU является то, что он не является плавной функцией, что делает его непригодным для некоторых алгоритмов оптимизации, которые полагаются на плавность, таких как метод сопряженного градиента. Наконец, ReLU не симметрична вокруг нуля, что может привести к тому, что выход всех нейронов будет иметь одинаковый знак, что приведет к замедлению сходимости и нестабильности обучения. Проблема "умирающего ReLU" решается одной из вариаций данной функции – Leaky ReLU.

$$\text{LeakyReLU}(x) = \max(0.01x, x)$$

В данной работе представлена ее модификация с добавлением утечки сверху. Leaky ReLU добавляет небольшой наклон к отрицательной области функции, позволяя градиентам по-прежнему проходить через нее и обновлять веса нейронов. Еще одним преимуществом Leaky ReLU является то, что это гладкая функция, что делает ее подходящей для алгоритмов оптимизации, которые полагаются на гладкость, таких

как метод сопряженного градиента. Более того, Leaky ReLU также не страдает от проблемы насыщения в положительной области, которая может привести к тому, что градиенты становятся очень маленькими и замедляют обучение (рис. 12). К ограничениям Leaky ReLU относятся непоследовательные предсказания для отрицательных входных значений и небольшой градиент в отрицательной области, который может замедлить обучение параметров.

$$f'(x) = \begin{cases} 0.01, & \text{если } x < 0; \\ 1, & \text{если } 0 \leq x \leq 1; \\ 0.01, & \text{если } x > 1. \end{cases}$$

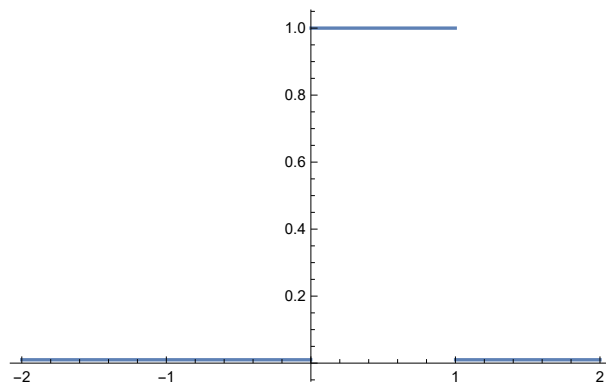


Рис. 12. График производной функции Mod Leaky ReLU

Получается для данной задачи лучше всего подойдет функция Leaky ReLU, что и подтверждается результатами обучения на тестовой выборке — 92.58%.

Заключение

В данной работе была реализована, обучена и протестирована математическая модель многослойного персептрона для распознавания рукописных цифр на основе набора данных MNIST. В процессе разработки были рассмотрены различные функции активации, а также проведен сравнительный анализ их эффективности. Практическая часть исследования продемонстрировала, что выбор функции активации играет ключевую роль в процессе обучения сети и точности предсказаний.

Полученные результаты показывают, что даже относительно простая архитектура нейронной сети при грамотной настройке может достигать высокой точности классификации. Данная работа может быть использована как основа для дальнейшего расширения возможностей модели, например, путем увеличения глубины сети, применения сверточных нейронных сетей или использования продвинутых методов оптимизации.

Таким образом, проведенное исследование подтвердило эффективность нейронных сетей в задачах распознавания образов и продемонстрировало важность выбора оптимальных параметров модели для достижения наилучших результатов.

Список использованных источников

1. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение. — 2-е изд. — М.: ДМК Пресс, 2018. — 652 с.
2. A Step by Step Backpropagation Example // mattmazor URL: <https://mattmazor.com/2015/03/17/a-step-by-step-backpropagation-example/> (дата обращения: 10.01.2025).
3. Нейронные сети для начинающих. Часть 1 // Habr URL: <https://habr.com/ru/articles/312450/> (дата обращения: 10.01.2025).
4. Функции активации нейросети // neurohive URL: <https://neurohive.io/ru/osnovy-data-science/activation-functions/> (дата обращения: 25.01.2025).